

2992702
Maditsi Kamogelo
Research Project
BLASTSMART APP PROGRAMMING

Dr. Milka Madahanna Mr. Tebatso Phala

July 2026

1 Abstract

This project developed a Python program called **BlastSmart** to help blasting engineers with blast calculations. The program has two calculators: the Surface Blasting Design Calculator and the Fragmentation Calculator. The program uses information such as the diameter of the blast-hole, the height of the bench, the type of rock, the explosive density, and the powder factor. Calculate the burden, spacing, stemming length, subdrill, actual powder factor, and mean fragment size. Algorithms, flowcharts, and pseudocode were used before writing Python code. The program can save time and reduce errors caused by manual calculations.

1. **Introduction** Dwala Global Resources is creating a mobile app called BlastSmart to help blasting engineers in South African mines. The app will make blast design calculations easier and faster than using paper books. The aim of this project is to build a simple mobile calculator that helps engineers do blast calculations quickly and correctly. The engineer will enter information such as bench height, burden, spacing, blast hole diameter, and other blast details. The app will then calculate the amount of explosive needed, the actual powder factor, the volume of rock, stemming length, and other important blast values. The objectives are to make blast calculations easy, reduce mistakes, save time, and help engineers plan safer and better blasting operations.

2. Methodology

- The algorithms for the Surface Calculator and Fragmentation Calculator were first written in simple English. This helped to explain the steps that the calculator must follow to solve each problem.
- After that, the algorithms were also presented in the form of a flowchart. The flowchart shows the order of the steps and how the program makes

decisions. The algorithm and flowchart were then used to write a pseudocode.

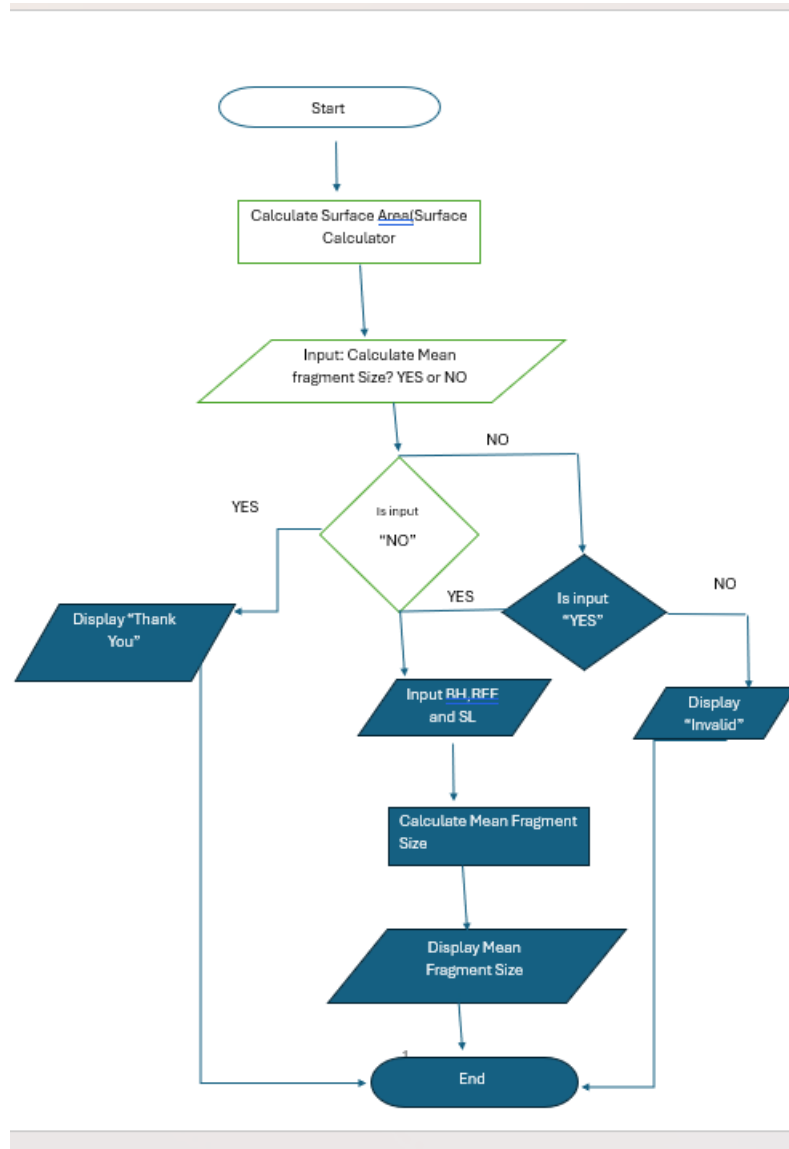
- After all this, a working python code was written, with the aid of the algorithms, flowcharts and pseudocode. The code was tested by entering different values to make sure the calculations were correct.
- Any errors found in the program were fixed by changing the code and testing it again until it worked properly.
- The final program was able to calculate important blasting values such as burden, stemming length, explosive mass, actual powder factor, and mean fragment size.
- The completed Python program will be used in the next stage of the BlastSmart mobile application so that blasting engineers can do calculations quickly and easily on their mobile devices.

RESULTS

3. ALGORITHMS 3.1) Algorithm for Surface Blasting Design Calculator and Flow Chart

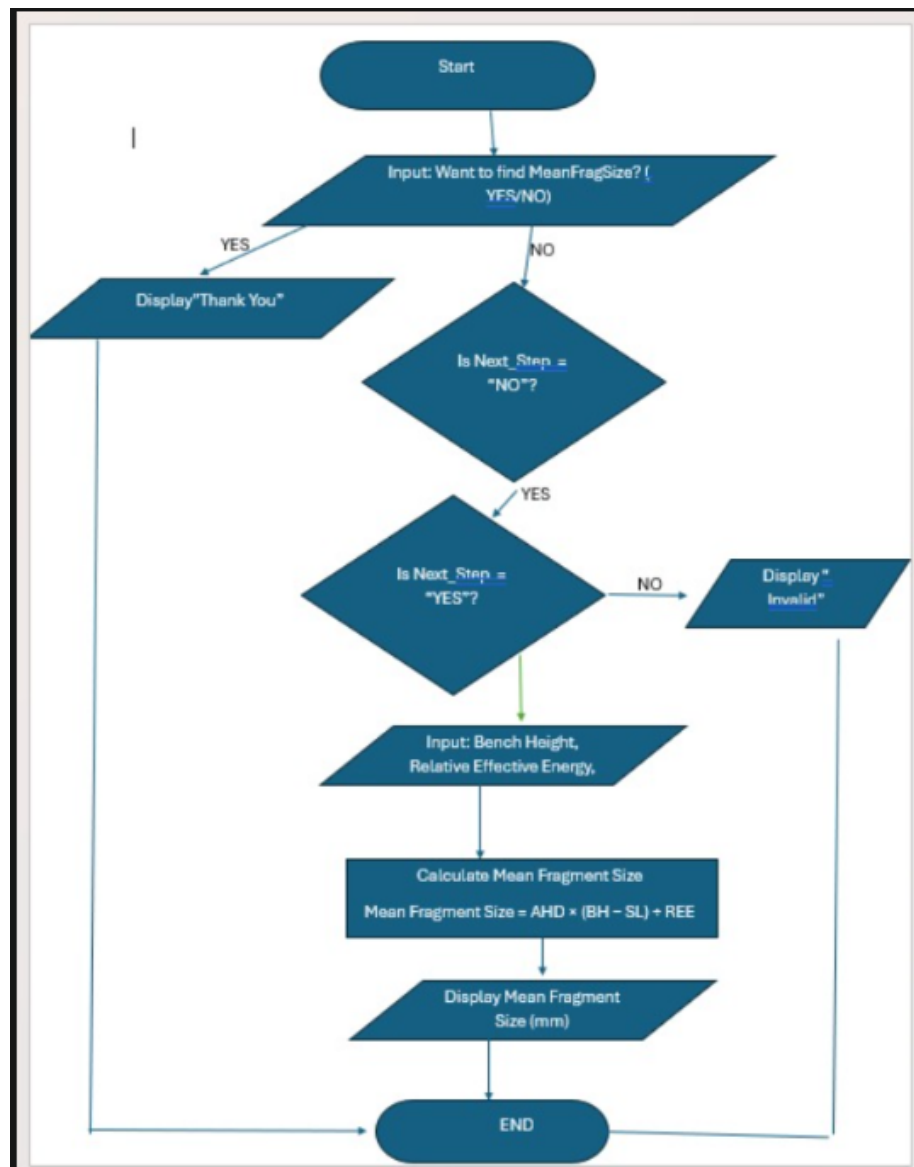
- Start the surface blasting calculator.
- Enter the blasting diameter.
- Enter the average in-hole density.
- Enter the spacing-to-burden ratio.
- Enter the bench height.
- Enter whether the blasting location is close to a community or not.
- Enter whether the rock type is hard or soft.
- Calculate the mass of explosive.
- Calculate the burden from the blasting diameter.
- Calculate the spacing using the spacing-to-burden ratio and burden.
- When the area is close to a community, calculate the stemming length using a factor of 30.
- When the area is not close to a community, calculate the stemming length using a factor of 20.
- Calculate the subdrill by dividing the burden by three.
- For hard rock, calculate the actual powder factor using the hard-rock formula.

- For soft rock, calculate the actual powder factor using the soft-rock formula.
- Display the burden, stemming length and actual powder factor.
- End the surface blasting calculation.



3.2) Algorithm for Fragmentation Calculator and Flow Chart

- Start the fragmentation calculator. Ask the user whether the mean fragment size must be calculated. When the answer is “NO”, display “Thank You”. When the answer is “YES”, enter the bench height. Enter the relative effective energy of the explosive. Enter the stemming length. Use the average in-hole density entered in the surface blasting calculator. Calculate the mean fragment size. Display the mean fragment size in millimetres. When the user enters an option other than “YES” or “NO”, display “Invalid”. End the program.
-



4. PSEUDOCODE

4.1 Surface Desing Calculator

– INPUT:

Surface Blasting Design Calculator: Blasting Diameter, Average In Hole Density, Spacing To Burden Ratio, Bench Height INPUT Proximity , Rock Type .

- Proximity TO UPPERCASE
- Rock Type TO UPPERCASE
-

$$Burden = \sqrt{\frac{MassofExplosive(BenchHeight - StemmingLength)}{Spacing - to - BurdenRatio \times TechnicalPowderFactor}} \quad (1)$$

$$MassofExplosive = \frac{AverageIn - HoleDensity \times (BlastingDiameter)^2}{1273} \quad (2)$$

$$Spacing = Spacing - to - BurdenRatio \times Burden \quad (3)$$

- Proximity = "CLOSE TO COMMUNITY"
- Stemming Length = 30 × (Blasting Diameter)
- IF Proximity = "NOT"
- Stemming Length = 20 × (Blasting Diameter)
- IF Subdrill = Burden ÷ 3
- Rock Type = "HARD" THEN Actual Powder Factor = (Bench Height - Stemming Length + Subdrill) ÷ (Burden × Spacing × Bench Height)
- IF RockType = "SOFT" THEN Actual Powder Factor = (Bench Height - Stemming Length - Subdrill) ÷ (Burden × Spacing × Bench Height)
END IF OUTPUT Burden OUTPUT Stemming Length OUTPUT Actual Powder Factor
- Surface Blasting Design Calculator

4.2 Fragmentation

•

BEGIN Fragmentation_Calculator

INPUT Next Step CONVERT Next Step TO UPPERCASE

IF Next Step = "NO" THEN OUTPUT "Thank You!"

IF Next Step = "YES" INPUT Bench Height

INPUT Relative Effective Energy Of Explosive INPUT Stemming Length 2
Mean Fragment Size = (Average In Hole Density × (Bench Height - Stemming Length 2)) ÷ Relative Effective Energy Of Explosive

OUTPUT Mean Fragment Size in "mm"

ELSE OUTPUT "Invalid!"

END IF

END Fragmentation Calculator

5. CODE

5.1. Surface Calculator

```
import math
BlastingDiameter = float(input("EnterBlastingDiameter(mm) : "))AverageInHoleDensity =
float(input("EnterAverageIn-HoleDensity(kg/m3) : "))SpacingToBurdenRatio =
float(input("EnterSpacing : BurdenRatio : "))BenchHeight = float(input("EnterBenchHeight(m) :
"))
Proximity = input("Enter 'Close to Community' or 'Not':
").upper() RockType = input("EnterRockType(Hard/Soft) : ").upper()TechnicalPowderFactor =
float(input("EnteryourTechnicalPowderFactor : "))
Massofanexplosive = (AverageInHoleDensity*BlastingDiameter**2)/1273
if Proximity == "CLOSE TO COMMUNITY": StemmingLength = 30 *
BlastingDiameterBurden = math.sqrt((Massofanexplosive*(BenchHeight-
StemmingLength))/(SpacingToBurdenRatio * TechnicalPowderFactor))
elif Proximity == "NOT": StemmingLength = 20*BlastingDiameterBurden =
math.sqrt((Massofanexplosive*(BenchHeight-StemmingLength))/(SpacingToBurdenRatio*
TechnicalPowderFactor))
else: print("Invalid Proximity.")
Spacing = SpacingToBurdenRatio * Burden
subdrill = Burden / 3
if RockType == "HARD" : ActualPowderfactor = (BenchHeight-StemmingLength+
subdrill)/(Burden*Spacing*BenchHeight)elif RockType == "SOFT" : ActualPowderfactor =
(BenchHeight-StemmingLength-subdrill)/(Burden*Spacing*BenchHeight)
else: print("Invalid Rock Type.")
print(" = " * 10, "OUTPUT", " = " * 10) print("Burden = ", Burden, "m")
print("Spacing = ", Spacing, "m") print("Stemming Length = ", StemmingLength, "m")
print("Actual Powder Factor = ", ActualPowderfactor)print(" = " * 10, "END", " =
" * 10)
```

5.2. Fragmentation Calculator

```
NextStep = input("WanttofindMeanFragSize(YES/NO) : ").upper()if NextStep ==
"NO" :
print("Thank You!")
elif NextStep == "YES" : BenchHeight = float(input("EnteryourBenchHeight :
"))
RelativeEffectiveEnergyofExplosive = float(input("EnteryourRelativeeffectiveenergyofexplosive :
"))StemmingLength2 = float(input("EnteryourStemmingLength : "))
MeanFragmentSize = (AverageInHoleDensity*(BenchHeightStemmingLength2))/
(RelativeEffectiveEnergyofExplosive)print(" MeanFragmentSize = ", MeanFragmentSize, "mm")
else: print("Invalid!")
```

6).COMBINATION OF THE CODES

```
import math
```

```

    BlastingDiameter = float(input("EnterBlastingDiameter(mm) : "))AverageInHoleDensity =
    float(input("EnterAverageIn-HoleDensity(kg/m3) : "))SpacingToBurdenRatio =
    float(input("EnterSpacing : BurdenRatio : "))BenchHeight = float(input("EnterBenchHeight(m) :
    "))
    Proximity = input("Enter 'Close to Community' or 'Not': ").upper()
    RockType = input("EnterRockType(Hard/Soft) : ").upper()TechnicalPowderFactor =
    float(input("EnteryourTechnicalPowderFactor : "))
    Massofanexplosive = (AverageInHoleDensity*BlastingDiameter**2)/1273
    if Proximity == "CLOSE TO COMMUNITY": StemmingLength = 30 *
    BlastingDiameterBurden = math.sqrt((Massofanexplosive*(BenchHeight -
    StemmingLength))/(SpacingToBurdenRatio * TechnicalPowderFactor))
    elif Proximity == "NOT": StemmingLength = 20*BlastingDiameterBurden =
    math.sqrt((Massofanexplosive*(BenchHeight - StemmingLength))/(SpacingToBurdenRatio*
    TechnicalPowderFactor))
    else: print("Invalid Proximity.")
    Spacing = SpacingToBurdenRatio * Burden
    subdrill = Burden / 3
    if RockType == "HARD" : ActualPowderFactor = (BenchHeight - StemmingLength +
    subdrill)/(Burden*Spacing*BenchHeight)elif RockType == "SOFT" : ActualPowderFactor =
    (BenchHeight - StemmingLength - subdrill)/(Burden*Spacing*BenchHeight)
    else: print("Invalid Rock Type.") exit()
    print(" = " * 10, "OUTPUT", " = " * 10) print("Burden =", Burden, "m")
    print("Spacing =", Spacing, "m") print("Stemming Length =", StemmingLength, "m")
    print("Actual Powder Factor =", ActualPowderFactor)print(" = " * 10, "END", " =
    " * 10)
    NextStep = input("WanttofindMeanFragSize(YES/NO) : ").upper()if NextStep ==
    "NO" : print("ThankYou!")elif NextStep == "YES" :
    BenchHeight = float(input("EnteryourBenchHeight : "))RelativeEffectiveEnergyofExplosive =
    float(input("EnteryourRelativeeffectiveenergyofexplosive : "))StemmingLength2 =
    float(input("EnteryourStemmingLength : "))MeanFragmentsSize = (AverageInHoleDensity*
    (BenchHeight - StemmingLength2))/(RelativeEffectiveEnergyofExplosive)
    print("Mean Fragment Size=", MeanFragmentsSize, "mm")else : print("Invalid!")

```

7. DISCUSSION

The Surface Blasting Design Calculator was created using Python to make blast design calculations easier and faster. The user enters the blasting diameter, average in-hole density, spacing-to-burden ratio, bench height, proximity to the community, rock type, and technical powder factor. The program then uses these values to calculate the mass of the explosive, stemming length, burden, spacing, subdrill, and the actual powder factor. The program uses if and elif statements to make decisions. If the blast is close to a community, the program uses a longer stemming length. If the blast is not close to a community, it uses a shorter stemming length. The program also checks whether the rock is hard or soft and calculates the actual powder factor using the correct equation. The calculator makes blast design easier because the user does not need to do the calculations by

hand. This helps to save time and reduce calculation mistakes. However, the program only gives correct results when the user enters the correct values. If incorrect or unrealistic values are entered, the program may produce invalid results or calculation errors. Overall, the program shows that Python can be used to solve blasting calculations quickly and simply. In the future, the calculator can be improved by adding more calculations, checking that the user enters correct values, and giving clear error messages when the input is incorrect. This will make the BlastSmart application more useful and reliable for blasting engineers.

8. CONCLUSION

The BlastSmart program was successfully developed using Python. It can calculate important blasting values quickly and easily. The program can help engineers reduce calculation mistakes and save time during blast planning. However, correct results depend on the user entering correct information. In the future, the program should include better error messages, input checking and more blasting calculations. It can also be developed into a simple mobile application for blasting engineers.